Campus Supply Desk - Checkpoint A

COMP 3415 | Software Engineering | Santhosh Allada

1. System Understanding Note

Campus Supply Desk is a Python terminal application for ordering campus supplies. A user can view products and stock, get a quote, place an order, view order history, look up an order, or cancel it. The application offers both an interactive menu and individual terminal commands. A quote shows what an order would cost. Placing an order deducts stock and saves an order record. README.md, sections 1-2

Component	Responsibility
supplydesk/cli.py, lines 11-67	Parses terminal commands and prints their results as JSON. Opens the interactive menu when the user supplies no command.
supplydesk/menu.py, read_request() and run_menu(), lines 25-89	Reads menu choices and order details, displays prices in dollars, and asks for confirmation before placing or cancelling an order.
supplydesk/service.py, lines 6-148	OrderService checks requests and stock, calculates prices, and handles placement, lookup, history, and cancellation.
supplydesk/storage.py, lines 7-29	JsonStore loads and saves application state. When there is no save file, it loads the starting products from catalog.json and initializes empty order history.

Execution flow - quoting two notebooks. The user enters python3 -m supplydesk quote --item notebook:2. The CLI's parse_item() converts notebook:2 into a product ID and an integer quantity. build_parser() supplies the defaults regular and pickup, and main() passes the request to OrderService.quote(). The service loads the catalog, finds the notebook, checks the requested quantity against available stock, and calculates the total. Two notebooks at 500 cents each give a total of 1000 cents. The CLI prints the quote as JSON. In a run against a fresh temporary path, the result was total_cents: 1000, and the quote created no save file. References: supplydesk/cli.py, parse_item(), build_parser(), main(); supplydesk/service.py, quote().

Hardest part to follow - OrderService.place_order(). This method handles several steps in one place. It validates the request, finds products, calculates prices, builds the order record, updates stock, and saves the result. The point where it starts changing state is easy to miss among the validation and pricing steps. To check that a quote and a placed order use the same pricing rules, a reader has to compare the two methods line by line. OrderService.place_order()

2. Five Code Problem Diagnoses

1. Duplicated code in quote() and place_order()

Evidence and consequence: Both methods repeat customer, fulfillment, and item validation as well as pricing. A change made in only one copy could make the quote and placed order disagree.

Proposed improvement: Share the validation and pricing logic. Keep stock deductions and saving in place_order().

2. Magic numbers in service.py, lines 47-52

Evidence and consequence: The amounts 5000, 10000, and 600 appear directly in pricing calculations. A reader has to infer which amount represents the student discount threshold, the free delivery threshold, or the delivery fee. This readability issue would remain even after consolidating the duplicated logic.

Proposed improvement: Give the amounts names that state their purpose and units. Keep the comparisons and values the same during this refactor.

3. Long method in menu.py, run_menu()

Evidence and consequence: One method displays the menu, dispatches every option, handles confirmations, and reports errors. Following the placement flow requires reading past branches for other actions.

Proposed improvement: Extract placement and cancellation into focused functions while preserving the prompts, confirmations, and error handling.

4. Testing gap in tests/test_smoke.py, lines 17-41

Evidence and consequence: All four starter tests exercise successful requests. None checks whether a rejected placement leaves stock, order history, and order numbering intact. A regression in invalid-request handling could pass the suite.

Proposed improvement: Add a rejected-placement test that checks both the error and its effect on application state.

5. Repeated cancellation restores stock twice in service.py, cancel_order()

Evidence and consequence: The method restores each item's quantity without checking whether the order is already cancelled. The observed notebook stock sequence was 20 -> 18 -> 20 -> 22 after placing an order for two and cancelling it twice. The functional specification's Cancellation section requires the second call to return the existing cancelled order without changing stock, prices, or history.

Proposed improvement: If the status is already cancelled, return the saved order before restoring stock or saving again. Record this as a defect fix separate from the behavior-preserving refactorings.

3. Short Initial Testing Strategy

Category	Input and expected result
Normal	With a fresh save path, place a regular pickup order for two notebooks. Expect ORD-0001, status placed, a total of 1000 cents, and notebook stock decreasing from 20 to 18.
Boundary	Quote regular standard delivery for two hoodies and four notebooks. The subtotal is 10000 cents, so expect a delivery fee of 0 cents and a total of 10000 cents. The spec makes standard delivery free when the subtotal after discount is at least 10000 cents. FUNCTIONAL_SPEC.md, Prices section
Invalid	Start with a fresh temporary path and no save file. Attempt to place a pickup order for zero notebooks. Expect ValueError, notebook stock still at 20, empty order history, and no save file. Place a valid order for one notebook afterward and check that it receives ORD-0001. Reopen the saved state and check that it contains exactly that order and 19 notebooks in stock.
Behavior to preserve	On a fresh state, quote and then place a student standard-delivery order for two hoodies. Check that both contain the same item lines and independently assert the expected subtotal of 8000 cents, discount of 800 cents, delivery fee of 600 cents, and total of 7800 cents.

Supporting execution evidence: python3 -m unittest discover -s tests -v reported Ran 4 tests and OK. The notebook quote returned total_cents: 1000 and created no save file. The cancellation reproduction used OrderService.place_order() followed by two cancel_order() calls with the same order ID and printed [20, 18, 20, 22]. The four checks in the strategy table are proposed Task 2 tests.